wsdm-cup-multilingual-chatbot-arena

5th Place Solution for the WSDM Cup - Multilingual Chatbot Arena

Rank	5
Team	Just doing
Author	Albert Wang
Topic ID	567724
Version	Original

Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/567724
Retrieved with Kaggle CLI on 2026-10-21.

Congratulations to all the winners!

Thanks to the organizers for hosting such a fun competition, and to UC Berkeley SkyLab, a16z, Sequoia, Fireworks AI, Together AI, RunPod, Anyscale, Replicate, Fal AI, Hyperbolic, Kaggle, MBZUAI, HuggingFace for generous sponsoring Chatbot Arena.
This competition is valuable and has aroused my interest because organizations can use models to provide answers that users like. Users' time is valuable, and how to predict their preferences and provide information that they are interested in can provide more value to users by improving the efficiency of information delivery.

Context

- Business context: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena
- Data context: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/data

Overview of the approach

Our final submission use Qwen2.5-14B. This model evaluates two LLM responses (A and B) to a user query, determining which is "better." It first checks for the vllm library and installs it if missing. A system prompt guides a judge LLM (Qwen2.5_14b) to assess responses based on criteria like completeness and clarity. The code preprocesses input, truncating long texts to fit within the model's token limit. It uses vllm to generate log probabilities for tokens 'A' and 'B', indicating preference. A post-processing step refines the decision: if log probabilities are close, it counts numbers in responses as a tie-breaker. Finally, the code outputs the "winner" (model_a or model_b) to a CSV file.

Progress Roadmap

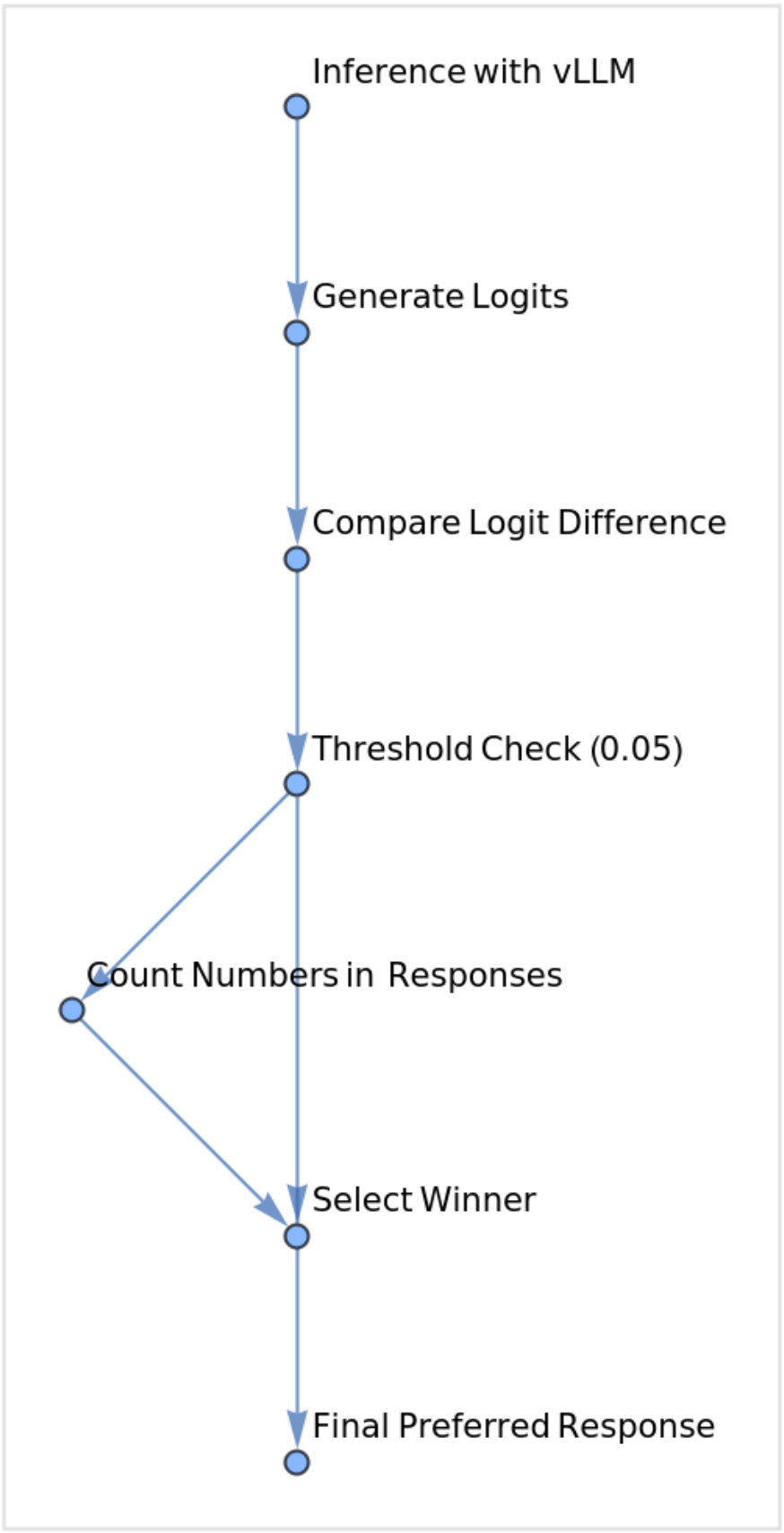
- 0.661 Start with Gemma-2 9b 4-bit QLoRA. We started with Gemma-2 9B running in 4-bit QLoRA, a solid baseline due to its efficiency in fine-tuning large language models with minimal GPU memory usage.
- 0.665 Applied post-processing, pick the winner while llm generated result is too close which is not reliable. Initial LLM-generated results were unreliable when scores were too close. To address this, we introduced a post-processing step where, if the log probability difference between two responses was below a certain threshold, we applied heuristic rules to determine the winner.
- 0.685 Ensemble with LGBM. To further boost accuracy, we ensembled the LLM predictions with a LightGBM model, leveraging handcrafted features such as token length, semantic similarity, and response coherence.
- 0.690 vLLM _ Qwen2.5-14B. We upgraded the base LLM model to Qwen2.5-14B, a stronger model with better multilingual capabilities and instruction-following accuracy.
- 0.693 Truncate tokens when overpass max_len, give priority to head and tail which are the parts users pay more attention to. Long responses often exceeded the model's max_length limit. To handle this, we truncated tokens while prioritizing the beginning and end of responses, as these sections typically contain the most valuable information.
- 0.697 swap response location. In some cases, responses were structured suboptimally. By swapping response locations in the input formatting, we allowed the model to evaluate the context better, improving ranking accuracy.

- 0.703 add wsdm_lmsys data. We integrated additional WSDM_LMSYS training data, enriching the dataset with diverse chatbot interactions and user preferences.
- 0.706 soft_label_swap (generate probability label instead of binary). Instead of treating response selection as a binary task, we generated probability-based labels for ranking, allowing the model to make more nuanced decisions.
- 0.706 ➡️ 0.712 public ➡️ private testing set

Code

https://github.com/datahubber/Natural-Language-Processing/blob/main/ChatbotArena/vllm-qwen2-5-14b-2ef57b.ipynb

postprocessing



vllm generate prediction results in logits, when the differences is lower than 0.05, it can be converted to 2.5% winning chance difference in probability. More numbers relate to more accurate the response which might be

preferred by users. Number count function works when llm is hard to decide. For example, vllm predict response a has 80% winning chance while b has 20%, the prediction of a is probably right. But when vllm predict response a has 51% and b has 49%, vllm has no strong confidence, in this case, single rule which has 0.58 accuracy improve overall solution accuracy.
Idea came from 《What Is ChatGPT Doing … and Why Does It Work》 by Stephen Wolfram, he thinks numbers are more accurate than language. So I implemented this method in model and found it works.

Things to try

1. Label user as logical/emotional ∈ [0, 1], logical ratio + emotional ratio = 1 according to prompt he/she made. And label response a/b with logical/emotional ratio ∈ [0, 1], logical ratio + emotional ratio = 1 for user-content match while llm prediction is vague.

Supplemental Q&A

> **Jianbo Wang · 2025-03-12T02:33:34.727000**
>
> Congratulations on the gold!
> I also applied heuristic rule but simply based on the length of responses. It seems your solution is more reasonable.
> If I'm not mistaken, is "soft_label_swap" a method similar to label smoothing (e.g., [0, 1] -> [0.1, 0.9])?

> **Chew Kok Wah · 2025-03-12T04:36:28.170000**
>
> Is your final 14B model also trained with 4bit QLora?

> **Albert Wang · 2025-03-12T11:20:23.740000**
>
> No, it didn't

> **Albert Wang · 2025-03-12T11:24:00.080000**
>
> Yes, partially correct. It includes smoothing (e.g., [0, 1] -> [0.1, 0.9]) and also swap position to enhance robustness.